

Organize Kubernetes pods using Labels and Namespaces

Step 1 - Initialize minikube

- Run *minikube start* and *minikube status* to start and check the status of minikube

```
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:30:32) -> minikube start
minikube v1.37.0 on Debian bookworm/sid
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.48 ...
Restarting existing docker container for "minikube" ...
Failing to connect to https://registry.k8s.io/ from inside the minikube container
To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
  Using image docker.io/kubernetes/dashboard:v2.7.0
  Using image docker.io/kubernetes/metrics-scraper:v1.0.8
Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

Enabled addons: storage-provisioner, default-storageclass, dashboard
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:30:59) -> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Step 2 - Write namespaces.yaml

```
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:29:52) -> cat namespaces.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: prod
```

Step 3 - Write pods.yaml

```
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:46:17) -> cat pods.yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: dev
  labels:
    app: frontend
    env: dev
spec:
  containers:
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: dev
  labels:
    app: backend
    env: dev
spec:
  containers:
    - name: backend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: prod
  labels:
    app: frontend
    env: prod
spec:
  containers:
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: prod
```

Step 4 - Apply YAML files

```
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:36:02) -> kubectl apply -f namespaces.yaml
namespace/dev unchanged
namespace/prod unchanged

(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:36:45) -> kubectl apply -f pods.yaml

pod/frontend-pod created
pod/backend-pod created
pod/frontend-pod created
pod/backend-pod created
```

Step 5 - Get pods by namespace

```
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:36:47) → kubectl get pods -n dev
NAME          READY   STATUS    RESTARTS   AGE
backend-pod   1/1     Running   0           52s
frontend-pod  1/1     Running   0           52s

(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:37:39) → kubectl get pods -n prod
NAME          READY   STATUS    RESTARTS   AGE
backend-pod   1/1     Running   0           56s
frontend-pod  1/1     Running   0           56s
```

Step 6 - Get pods by Labels

```
(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:38:16) → kubectl get pods -n prod -l app=frontend
NAME          READY   STATUS    RESTARTS   AGE
frontend-pod  1/1     Running   0           102s

(~/Desktop/devops/lab9) (1rv24mc038_shreerakshahr@asus:pts/0)
(09:38:29) → kubectl get pods -n dev -l app=backend
NAME          READY   STATUS    RESTARTS   AGE
backend-pod   1/1     Running   0           112s
```